

Dashboard ejecutivo de métricas de ventas

Permite a un Jefe de Ventas revisar en ~5 minutos el estado de su pipeline y navegar entre cuatro escenarios de datos (A, B, C, D).

```
cd backend && npm install && npm run dev    # http://localhost:3001
cd frontend && npm install && npm run dev    # http://localhost:5173
```

1. Decisiones técnicas

Express.js: Se descartaron frameworks más completos por no aportar nada a un problema de pura lectura. Middleware seleccionado con el mismo criterio: `compression` reduce los 668 KB del dataset a ~80 KB en tránsito; `helmet`, `morgan` y `express-rate-limit` cubren seguridad y observabilidad sin infraestructura adicional.

Vite: El frontend llama a `/api` sin configurar CORS en desarrollo. **Recharts** sobre `Chart.js` o `D3` por ser declarativo y nativo en React. **Vanilla CSS con custom properties** mantiene la consistencia visual sin dependencias externas. Sin librería de estado global: el estado de la app son dos variables que `useState` maneja sin necesidad de `Zustand` ni `Redux`.

Estructura de directorios por responsabilidad: Tanto backend como frontend comparten las mismas capas (`config/`, `interfaces/`, `services/`, `controllers|hooks/`, `handlers|utils/`, `routes|pages/`). La ventaja es predictibilidad: un bug en la capa HTTP se busca en `controllers/`; uno en los datos, en `services/`. El frontend espeja la estructura del backend para reducir la fricción al navegar entre ambos.

Stale-while-revalidate y panel de resumen: Al cambiar de dataset, los hooks mantienen los datos anteriores visibles mientras llega la respuesta nueva. El panel de resumen semanal (`utils/insights.ts`) es una función pura que clasifica métricas por severidad y genera observaciones de negocio ordenadas por prioridad.

2. Segunda iteración

- **Selector de rango de fechas:** El backend ya acepta `?from=&to=`; solo falta el control en el frontend.
- **Mini sparklines en KPI cards:** Una línea de 30 puntos dentro de cada card daría contexto visual que el porcentaje solo no entrega.
- **Modo comparación:** Superponer dos datasets en el mismo gráfico para entender las diferencias entre escenarios.
- **Persistencia de preferencias:** Guardar dataset activo y métrica en `localStorage`.
- **Tests y datos en tiempo real:** `insights.ts` y `metrics.service.ts` son candidatas a unit tests por ser funciones puras; en producción el backend reemplazaría el JSON por una conexión a base de datos.